Lean Construction Attendance System

Full Technical Architecture, Database Design & Integration Documentation

---

Project: Lean Construction Module 1	**Database:** Supabase (PostgreSQL 17)
Frontend: React (Vite) + Vanilla CSS	**Hardware:** ESP32 + R307 Fingerprint Sensor
Hosting: Vercel (Web) + Supabase Edge	**Status:** Fully Built & Verified

1. Executive Summary & Problem Overview

Traditional construction site attendance relies on manual paper registers or basic supervisor roll calls. This legacy approach causes proxy attendance (buddy punching), delayed reporting to contractors, payroll disputes, and zero real-time visibility into active on-site workforce numbers.

The Solution: An integrated Internet-of-Things (IoT) biometric attendance ecosystem. Workers scan their fingerprint at the site gate via an ESP32 microcontroller with an R307 optical fingerprint sensor. The hardware securely transmits the scan over WiFi to a Supabase cloud database, which instantly updates role-based web dashboards (Contractor, Supervisor, Worker) hosted on Vercel via WebSocket Realtime sync.

2. Complete Technology Stack

Layer	Technologies Used	Purpose
Frontend UI	React (Vite), HTML5, Modern Vanilla CSS	Responsive web portal for Contractors, Supervisors, and Workers.
Database	Supabase PostgreSQL 17	Relational cloud DB with Row Level Security (RLS) & triggers.
Authentication	Supabase Auth (GoTrue API)	Secure email/password authentication & JWT session management.
Backend API	Supabase Edge Functions (Deno Runtime)	Serverless HTTP relay for ESP32 hardware authentication & check-in logic.
Hardware	ESP32 Board, R307 Optical Sensor, C++	On-site physical fingerprint scanner & WiFi HTTP client.
Deployment	Vercel (Frontend), Supabase Cloud, GitHub	Continuous deployment & version control.

3. Architectural Choice: Why Supabase Over Firebase?

Selecting the right backend was a critical architectural decision. Here is why Supabase (PostgreSQL) was chosen over Google Firebase (Firestore NoSQL):

- **Relational Data Structure (PostgreSQL vs. NoSQL Document Store):** Attendance data is strictly relational: *Workers* have multiple *Attendance Logs*, which belong to specific *Sites* and *Roles*. Postgres handles foreign keys, SQL joins, and relational integrity natively. Firebase Firestore relies on flat NoSQL collections, forcing redundant data duplication and expensive client-side joins.
- **Postgres Row-Level Security (RLS) vs. Firebase Security Rules:** Supabase leverages native PostgreSQL RLS policies defined directly at the database engine level. Even if a malicious user inspects network traffic and extracts the API keys, the database itself enforces that a worker can ONLY query rows where user_id = auth.uid().
- **Automated Database Triggers & Stored Procedures:** Supabase allows writing SQL triggers directly inside Postgres (e.g., auto-calculating present vs late status on insert based on 9 AM cutoff). Firebase requires running separate Cloud Functions for every trigger, adding extra latency and complexity.
- **No Vendor Lock-In & Open Standard SQL:** Supabase is built on standard open-source PostgreSQL. If needed, the entire database can be exported as a standard .sql file and hosted anywhere (AWS RDS, GCP Cloud SQL, or self-hosted Docker). Firebase locks your application into proprietary Google APIs.

4. Database Schema & Table Structure

The database schema consists of two core relational tables created in Supabase PostgreSQL:

Table A: public.users

Column	Type	Constraints / Notes
id	uuid (PK)	References auth.users(id) ON DELETE CASCADE
full_name	text	Worker / Contractor / Supervisor full name
role	text	CHECK (role IN ('contractor', 'supervisor', 'worker'))
fingerprint_id	integer	UNIQUE constraint. Maps to numeric slot on R307 sensor
created_at	timestamptz	DEFAULT now()

Table B: public.attendance

Column	Type	Constraints / Notes
id	uuid (PK)	DEFAULT gen_random_uuid()
user_id	uuid (FK)	References public.users(id) ON DELETE CASCADE
check_in	timestamptz	Timestamp when worker scanned finger at entry
check_out	timestamptz	Nullable. Filled when worker scans finger at exit
status	text	Auto-derived by trigger: 'present' or 'late'
device_id	text	Identifies which ESP32 scanner sent the log

5. Automated Database Triggers

To keep business logic consistent and reduce client-side code, two SQL triggers were created:

```
-- Trigger 1: Auto-derive 'present' vs 'late' status based on 9 AM IST cutoff
CREATE OR REPLACE FUNCTION public.derive_attendance_status()
RETURNS TRIGGER AS $$
BEGIN
  IF EXTRACT(HOUR FROM NEW.check_in AT TIME ZONE 'Asia/Kolkata') < 9 THEN
    NEW.status := 'present';
  ELSE
    NEW.status := 'late';
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_derive_attendance_status
  BEFORE INSERT ON public.attendance
  FOR EACH ROW EXECUTE FUNCTION public.derive_attendance_status();

-- Trigger 2: Auto-create public.users row on Auth signup
CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

6. End-to-End Data Flow (Hardware ➔ Cloud ➔ Web)

Here is step-by-step how a physical scan on the construction site updates the web portal in real time:

- **Step 1: Physical Fingerprint Scan:** Worker places finger on R307 optical sensor connected to ESP32 at site gate. The sensor matches the fingerprint template and returns numeric slot (e.g., ID 1).
- **Step 2: HTTP POST to Edge Function Relay:** ESP32 sends JSON payload { fingerprint_id: 1, device_secret: '...' } over WiFi to Supabase Edge Function attendance-relay.
- **Step 3: Serverless Business Logic:** Edge Function validates device secret, looks up worker UUID from users table, checks if check-in exists for today:
  - **No entry today:** INSERTS new check-in row (Trigger sets status = 'present' or 'late').
  - **Checked in, check_out IS NULL:** UPDATES row with check_out = now().
  - **Already checked out:** Returns response 'Already checked out today'.
- **Step 4: Supabase Realtime Publication:** PostgreSQL publishes the INSERT/UPDATE mutation to the supabase_realtime channel over WebSockets.
- **Step 5: React UI Dynamic Render:** React web portal subscription catches event instantly and updates stats cards & attendance tables without page refresh.

7. How Frontend Fetches & Listens to Supabase Data

The React web application uses `@supabase/supabase-js` client to fetch initial data and subscribe to WebSocket changes:

```
// 1. Initial Data Fetch (Contractor & Supervisor Dashboards)
const [attendanceRes, usersRes] = await Promise.all([
  supabase.from("attendance").select("*").order("check_in", { ascending: false }),
  supabase.from("users").select("*").eq("role", "worker")
]);
```

```
// 2. Realtime WebSocket Listener (Updates UI live on hardware scan)
useEffect(() => {
  const channel = supabase
    .channel("attendance-realtime")
    .on("postgres_changes", { event: "*", schema: "public", table: "attendance" },
      (payload) => {
        if (payload.eventType === "INSERT") {
          setAttendance((prev) => [payload.new, ...prev]);
        } else if (payload.eventType === "UPDATE") {
          setAttendance((prev) => prev.map((a) => a.id === payload.new.id ? payload.new : a));
        }
      }
    )
    .subscribe();

  return () => supabase.removeChannel(channel);
}, []);
```

8. Hardware Specifications & Sensor Wiring

The hardware module consists of an ESP32 Wi-Fi microcontroller connected to an R307 Optical Fingerprint Sensor.

R307 Wire Color	Function	ESP32 Pin Connection
■ Red	Power (VCC)	3.3V (or 5V)
■ Black	Ground (GND)	GND
■ Green	Serial Receive (RX)	GPIO 16 (RX2)
■ White	Serial Transmit (TX)	GPIO 17 (TX2)

9. Conclusion & Next Steps (Module 2+)

Module 1 successfully establishes a complete end-to-end attendance pipeline: physical fingerprint scan ➔ ESP32 transmission ➔ Supabase Edge authentication ➔ PostgreSQL relational trigger processing ➔ Vercel React Realtime web dashboard updates.

Upcoming Enhancements for Module 2:
• **Automated End-of-Day Absentee Cron Job:** Supabase scheduled Edge Function to mark non-checked-in workers as absent.
• **Remote Over-The-Air Fingerprint Registration:** Enabling supervisors to put the ESP32 into enrollment mode directly from the web app.
• **Construction Site Scoping (site_id):** Supporting multiple active construction sites under a single contractor account.